Internal Learning Portal – AI Build Specification

Living document. Every feature is added here, then handed to an AI coding agent to implement. Keep it precise, update the Changelog at the bottom when anything changes.

0. How the AI Agent Must Use This Document

1. Read sections 1–4 fully before writing any code. They define the stack, conventions, roles, and data model that apply to every feature.
2. Implement features **one at a time, in the order of Section 6**, unless told otherwise. Do not start a feature until its dependencies are done.
3. For each feature: create DB migration → RLS policies → backend routes → frontend pages → tests. Then tick the Definition of Done checklist.
4. If anything is ambiguous, **do not guess silently**: list assumptions at the end of your response, and pick the simplest option consistent with this document.
5. Never add public sign-up, social login, or features not listed here.
6. Do not change the conventions in Section 3 without being told to.

1. Project Overview

- **Type:** Internal-only web app for a school/academy (teachers and students, managed by admins).
- **Purpose:** Teachers share resources, assign work, run quizzes, and post announcements; students view courses, submit work, see marks, chat in class, and file complaints.
- **Access:** Invite/admin-created accounts only. There is **no sign-up page**.
- **Users:** Admin, Teacher, Student.

2. Tech Stack

Layer	Choice	Notes
Frontend	React 18 + Vite + TypeScript	React Router, TanStack Query for server state, React Hook Form + Zod for forms
UI	Tailwind CSS + shadcn/ui	Responsive, mobile-first, accessible

Layer	Choice	Notes
Backend	Node.js 20 + Express + TypeScript	REST JSON API under `/api/v1`, Zod validation on every input
Database	Supabase Postgres	SQL migrations in `/supabase/migrations`, Row Level Security ON for every table
Auth	Supabase Auth (email + password)	Sign-ups disabled in Supabase settings; users created by admin via service role
File storage	Supabase Storage	Private buckets, signed URLs for download
Realtime	Supabase Realtime	Used for classroom chat and announcements
Testing	Vitest (unit), Supertest (API), Playwright (a few e2e flows)	

2.1 Folder Structure

```
/apps
  /web            # React frontend
    /src
      /features   # one folder per feature (auth, courses, assignments, ...)
      /components # shared UI
      /lib        # supabase client, api client, utils
      /routes
  /api            # Node/Express backend
    /src
      /modules    # one folder per feature: routes, controller, service,
schema
      /middleware # auth, role guard, error handler, rate limit
      /lib
/supabase
  /migrations
  seed.sql
/docs
  SPEC.md         # this document
```

2.2 Environment Variables

- Web: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_API_URL`

- API: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_JWT_SECRET`, `PORT`, `CORS_ORIGIN`
- Never commit `.env`. Provide `.env.example` for both apps. The service role key is used **only** on the server.

3. Global Conventions

- **Language:** TypeScript strict mode everywhere.
- **API format:** success `{ "data": ... }`, error `{ "error": { "code": "STRING", "message": "human text" } }`. Use correct HTTP status codes.
- **Auth on API:** every route except `/auth/*` requires `Authorization: Bearer <supabase JWT>`. Middleware loads the user's profile and role.
- **Authorization:** role guard middleware (`requireRole('teacher')`) **and** Postgres RLS. Both must enforce access (defense in depth).
- **IDs:** UUID. **Timestamps:** `timestamptz`, UTC, columns `created_at`, `updated_at`.
- **Naming:** DB `snake_case`, TS `camelCase`, React components `PascalCase`.
- **Pagination:** list endpoints accept `?page=1&pageSize=20`, return `{ data, meta: { page, pageSize, total } }`.
- **Validation:** Zod schemas shared between client and server where possible.
- **Errors/UX:** every page has loading, empty, and error states. Forms show inline validation errors. Use toasts for success/failure.
- **Security:** helmet, CORS restricted to `CORS_ORIGIN`, rate limiting on auth endpoints, sanitize user text, file type and size limits on uploads (default max 20 MB; allowed: pdf, docx, pptx, xlsx, png, jpg, mp4, zip).
- **Accessibility:** keyboard navigable, labels on inputs, sufficient contrast.

4. Roles and Permissions

Capability	Admin	Teacher	Student
Create/disable users, reset any password	Yes	No	No
Create courses, assign students & teachers	Yes	No	No
Upload resources in own courses	Yes	Yes	No
View resources in enrolled/assigned courses	Yes	Yes	Yes

Capability	Admin	Teacher	Student
Create assignments/quizzes, grade	Yes	Own courses	No
Submit assignments / attempt quizzes	No	No	Yes
View own marks	–	–	Yes
View all marks of a course	Yes	Own courses	No
Post announcements	Yes	Own courses	No
Classroom chat	Yes (read)	Own courses	Enrolled courses
File a complaint	Yes	Yes	Yes
View/resolve all complaints	Yes	No	No

5. Core Data Model (shared by all features)

Create these in the first migration. Feature sections add their own tables.

`profiles`: `id` (uuid, PK, = `auth.users.id`), `full_name`, `email`, `role` (`admin|teacher|student`), `roll_number` (nullable, students), `is_active` (bool, default true), `must_change_password` (bool, default true), `created_at`, `updated_at`.

`courses`: `id`, `code` (unique), `title`, `description`, `teacher_id` (FK profiles), `is_active`, `created_at`, `updated_at`.

`enrollments`: `id`, `course_id` (FK), `student_id` (FK profiles), `enrolled_at`. Unique `(course_id, student_id)`.

Helper SQL functions (security definer): `current_role()`, `is_teacher_of(course_id)`, `is_enrolled_in(course_id)`. Use them in RLS policies.

6. Features (build in this order)

Feature 1 – Login (No Sign Up)

Goal: Authenticated access for internal users only.

User stories

- As a user, I log in with email and password.
- As an admin, I create user accounts (no public registration).

Requirements

- Page `/login`: email, password, "Forgot password?" link, submit button, error message on bad credentials.
- **No `/signup` route, no sign-up UI.** Disable public sign-ups in Supabase Auth settings.
- Admin-only endpoint `POST /api/v1/admin/users` creates a user via Supabase Admin API with `full_name`, `email`, `role`, temporary password, and inserts the `profiles` row. Also `PATCH /admin/users/:id` (edit, activate/deactivate) and `GET /admin/users`.
- After login, redirect by role: admin → `/admin`, teacher → `/teacher`, student → `/student`.
- If `must_change_password` is true, force redirect to change-password page before anything else.
- Deactivated users cannot log in or use existing sessions (check `is_active` in API middleware).
- Protected routes component; unauthenticated users go to `/login`. Session persists via Supabase session; logout button clears it.
- Rate limit login attempts (5 per minute per IP).

Acceptance criteria

- Valid credentials log in and land on the correct dashboard.
- Invalid credentials show a generic error (do not reveal whether the email exists).
- Visiting any protected URL logged out redirects to `/login`.
- A student cannot open `/teacher` or `/admin` routes (UI guard and API 403).

Feature 2 – Resource Uploading (Teacher) and Marks Viewing (Student)

Goal: Teachers share learning material per course; students access it and see their marks.

Tables: `resources`: `id`, `course_id`, `uploaded_by`, `title`, `description`, `file_path`, `file_name`, `file_size`, `mime_type`, `created_at`.

Requirements

- Teacher page `/teacher/courses/:id/resources`: upload form (title, description, file), list with download and delete.
- Student page `/student/courses/:id/resources`: list and download only.

- Files stored in private bucket `resources` at `{course_id}/{uuid}-{filename}`. Download via short-lived signed URL (60 seconds) generated by the API after permission check.
- Student page `/student/marks`: a summary of all marks across enrolled courses (pulls from Feature 7), grouped by course. Read-only.
- RLS: teachers insert/delete only in their courses; students select only for enrolled courses.

Acceptance criteria

- Teacher can upload, see, and delete resources only for their own courses.
- Student sees resources only for enrolled courses; direct file URL without a valid signed URL fails.
- Invalid file type or size over limit shows a clear error.

Feature 3 – Course Assignment to Students

Goal: Admin assigns courses to students (and a teacher to each course).

Requirements

- Admin pages: `/admin/courses` (CRUD), `/admin/courses/:id/students` (enroll/unenroll).
- Bulk enroll: multi-select students or paste a list of emails/roll numbers.
- Students see "My Courses" at `/student/courses`; teachers see "My Courses" at `/teacher/courses`.
- Endpoints: `GET/POST /courses`, `PATCH/DELETE /courses/:id`, `POST /courses/:id/enrollments`, `DELETE /courses/:id/enrollments/:studentId`.
- Unenrolling hides course content from the student but **keeps** their submissions and marks.

Acceptance criteria

- Enrolled student sees the course immediately; unenrolled student no longer does.
- Duplicate enrollments are rejected gracefully.

Feature 4 – Assignments

Tables: `assignments`: `id`, `course_id`, `title`, `instructions`, `due_at`, `max_marks`, `allow_late` (bool), `attachment_path` (nullable), `created_by`, `created_at`. `submissions`: `id`, `assignment_id`, `student_id`, `file_path`, `text_answer`, `submitted_at`, `is_late` (bool), `marks_obtained` (nullable), `feedback` (nullable), `graded_by`, `graded_at`. Unique `(assignment_id, student_id)`.

Requirements

- Teacher: create/edit/delete assignment, view submissions list per assignment, download files, enter marks and feedback.
- Student: see assignments per course with status (Not submitted / Submitted / Graded / Late), submit a file and/or text, resubmit until due date (or after, if `allow_late`).
- `is_late` computed server-side from `due_at`. Never trust client time.
- Marks must be between 0 and `max_marks`.
- Students can see only their own submission and grade.

Acceptance criteria

- Submission after due date is blocked unless `allow_late`, in which case it is flagged late.
- Teacher grading updates the student's view and Marks (Feature 7).

Feature 5 – Quizzes

Tables: `quizzes`: `id`, `course_id`, `title`, `instructions`, `start_at`, `end_at`, `duration_minutes`, `max_marks`, `shuffle_questions`, `is_published`, `created_by`. `quiz_questions`: `id`, `quiz_id`, `type` (`mcq_single|mcq_multi|true_false|short_answer`), `question_text`, `options` (jsonb), `correct_answer` (jsonb), `marks`, `position`. `quiz_attempts`: `id`, `quiz_id`, `student_id`, `started_at`, `submitted_at`, `answers` (jsonb), `score`, `status` (`in_progress|submitted|graded`). Unique `(quiz_id, student_id)` (one attempt).

Requirements

- Teacher builds quiz with a question editor, publishes it, and sees results per student.
- Student can start only between `start_at` and `end_at`; a countdown timer enforces `duration_minutes`; **server** enforces the deadline and rejects late answers.
- Auto-grade MCQ and true/false on submit. Short answers are marked manually by the teacher.
- **Never send `correct_answer` to the student client** before the quiz ends. Use a view or API serializer that omits it.
- Autosave answers every 30 seconds so a refresh doesn't lose progress.

Acceptance criteria

- A student cannot attempt twice, cannot start outside the window, and cannot see answers early.
- Auto-graded score appears in Marks immediately after submit.

Feature 6 – Announcements

Tables: `announcements`: `id`, `course_id` (nullable; null = global, admin only), `author_id`, `title`, `body`, `is_pinned`, `created_at`.

Requirements

- Teachers post to their courses; admins post globally or to any course.
- Students see a feed on their dashboard and per course; unread badge using `announcement_reads` (`announcement_id`, `user_id`, `read_at`).
- Realtime: new announcements appear without refresh.
- Pinned items first, then newest first.

Acceptance criteria

- Students only see announcements for enrolled courses plus global ones.
- Author or admin can edit/delete; others cannot.

Feature 7 – Marks (Assignments and Quizzes)

Goal: One consolidated view of all marks.

Requirements

- Build a database view `marks_overview` that unions graded `submissions` and graded `quiz_attempts`: `student_id`, `course_id`, `item_type` (`assignment|quiz`), `item_id`, `title`, `marks_obtained`, `max_marks`, `percentage`, `graded_at`.
- Student `/student/marks`: filter by course and type, per-course totals and overall percentage.
- Teacher `/teacher/courses/:id/gradebook`: table of students × items with totals; export CSV.
- Endpoint: `GET /marks/me`, `GET /courses/:id/marks` (teacher/admin).
- Only graded items count toward totals. Ungraded show as "Pending".

Acceptance criteria

- Totals match the sum of underlying records.
- A student can never query another student's marks (RLS verified by test).

Feature 8 – Classroom Chat

Tables: `chat_messages`: `id`, `course_id`, `sender_id`, `body` (max 2000 chars), `created_at`, `deleted_at` (soft delete).

Requirements

- One chat room per course, available to enrolled students and the course teacher.
- Realtime via Supabase Realtime subscription; messages load last 50, infinite scroll upward for history.
- Teacher/admin can delete any message; users can delete their own.
- Show sender name and role badge, timestamps, and auto-scroll to newest.
- Basic spam control: max 10 messages per 10 seconds per user.
- Text only for v1 (no file uploads).

Acceptance criteria

- Messages appear for all connected members within about 1 second.
- Non-members cannot read or subscribe to the room (RLS on Realtime).

Feature 9 – Register a Complaint

Tables: `complaints`: `id`, `submitted_by`, `category` (`academic|technical|facility|conduct|other`), `subject`, `description`, `attachment_path` (nullable), `status` (`open|in_review|resolved|rejected`), `admin_response`, `resolved_by`, `created_at`, `updated_at`.

Requirements

- Any logged-in user: form at `/complaints/new` and list of own complaints at `/complaints` with status.
- Admin: `/admin/complaints` with filters (status, category), detail view, set status, write response.
- Submitter sees status changes and the admin response; email notification optional (v2).
- Users can see only their own complaints; only admins see all.

Acceptance criteria

- Submitted complaint appears as `open` for the user and in the admin list.
- Status and response updates are visible to the submitter.

Feature 10 – Reset Password

Requirements

- "Forgot password?" on `/login` → `/forgot-password` (email field) → Supabase sends reset email → `/reset-password` page to set a new password (min 8 chars, validated).
- Always show the same confirmation message whether or not the email exists.

- Logged-in users can change password at `/settings/password` (current + new password).
- Admin can set a temporary password for a user, which sets `must_change_password = true`.
- Rate limit reset requests (3 per hour per email).
- Configure the Supabase redirect URL to the deployed `/reset-password` route.

Acceptance criteria

- Reset link works once and expires; old password stops working after reset.
- Weak passwords are rejected with a clear message.

7. Non-Functional Requirements

- **Performance:** list pages respond under 500 ms for typical data; index all foreign keys and frequently filtered columns.
- **Responsive:** fully usable on mobile (students will use phones).
- **Audit:** log grading changes and complaint status changes (who, when, old/new value) in an `audit_log` table.
- **Backups/Deployment:** frontend on Vercel/Netlify, API on Render/Railway/Fly, Supabase managed. Provide a README with setup and deploy steps.

8. Definition of Done (per feature)

- [ ] Migration with tables, indexes, and RLS policies
- [ ] API routes with Zod validation and role guards
- [ ] Frontend pages with loading, empty, and error states
- [ ] Tests: at least one API test for permissions and one for the happy path
- [ ] Acceptance criteria above verified
- [ ] Seed data updated (1 admin, 2 teachers, 5 students, 2 courses)
- [ ] Changelog updated

9. Out of Scope for v1

Public sign-up, social login, payments, video calls, mobile native apps, email notifications, file sharing in chat, plagiarism detection.

10. Open Questions (fill in before building)

- Institution name and branding (logo, colors)?
- Can one course have multiple teachers?
- Should students see class rank or only their own marks?
- Any grading scale (percentage, GPA, letter grades)?

11. Changelog

- v0.1: Initial spec with 10 features.